■ BankFlow

Domain-Driven Design — Full Project Scenario

NestJS · CQRS · Domain Events · Event-Driven Architecture

BankFlow is a digital banking backend built with Domain-Driven Design (DDD) principles. It demonstrates how to model complex financial business rules using Aggregates, Value Objects, Domain Events, and Bounded Contexts — all wired together with NestJS CQRS.

Actors

Actor	Role
Customer	Opens accounts, performs transactions (deposit, withdraw, transfer)
Bank Admin	Can freeze or close accounts
System	Auto-detects fraud, fires domain events, sends notifications

Bounded Contexts

1 · Identity & Customer Context

Responsibility: Register and manage bank customers.

Aggregate: Customer

Value Objects: Email, FullName, NationalId

Flow:

- Customer registers with name, email, and national ID.
- System validates no duplicate email or national ID.
- CustomerRegistered event is fired.
- Customer can update their profile at any time.

Domain Events:

CustomerRegistered	Fired after a new customer is successfully registered
CustomerUpdated	Fired when customer profile data changes

2 · Account Context

Responsibility: Manage bank account lifecycle.

Aggregate: BankAccount

Value Objects: IBAN, Currency, AccountStatus

Status Enum: ACTIVE | FROZEN | CLOSED

Flow:

- Registered customer requests to open an account (USD or EUR).
- System generates a unique IBAN.
- Account starts with 0 balance and ACTIVE status.
- AccountOpened event is fired.
- Admin can freeze an account (suspicious activity).
- Admin can close an account — only if balance is 0.
- One customer may hold up to 3 accounts.

Business Rules:

- ■ Cannot close an account with balance > 0.
- ■ Cannot transact on a FROZEN or CLOSED account.
- ■ One customer — maximum 3 accounts.

Domain Events:

AccountOpened	New account successfully created
AccountFrozen	Account frozen by admin or fraud system
AccountUnfrozen	Account reactivated after review
AccountClosed	Account permanently closed

3 · Transaction Context

Responsibility: Handle all money movement.

Aggregate: Transaction

Value Objects: Money, TransactionType

Type Enum: DEPOSIT | WITHDRAWAL | TRANSFER_IN | TRANSFER_OUT

Deposit Flow:

- Customer deposits money into an account.
- Balance increases.
- MoneyDeposited event is fired.

Withdrawal Flow:

- Customer requests a withdrawal.
- System checks sufficient balance and ACTIVE status.
- Balance decreases.
- MoneyWithdrawn event is fired.

Transfer Flow:

- Customer initiates transfer from Account A to Account B.
- System validates both accounts are ACTIVE.
- System checks sender has sufficient balance.

- Debit sender — TransferDebited event.
- Credit receiver — TransferCredited event.
- If anything fails — TransferFailed event, debit is rolled back.

Business Rules:

- Minimum transaction amount: $1.
- Maximum single withdrawal: $10,000.
- Maximum daily withdrawals: $20,000.
- Transfers between different currencies are not supported (v1).

Domain Events:

MoneyDeposited	Successful deposit completed
MoneyWithdrawn	Successful withdrawal completed
TransferDebited	Sender account debited in a transfer
TransferCredited	Receiver account credited in a transfer
TransferFailed	Transfer failed — rollback initiated

4 · Fraud Detection Context

Responsibility: Monitor transactions and flag suspicious behavior. This context only listens to events — it is never called directly.

Entity: FraudAlert

Trigger Rules:

- More than 5 withdrawals within 1 hour.
- Single transaction greater than $8,000.
- 3 failed transfers in a row.

Flow:

- Listens to: MoneyWithdrawn, TransferFailed, MoneyDeposited.
- Evaluates rules against the account's recent history.
- If a rule triggers — creates a FraudAlert record.
- Fires FraudDetected event.
- Account Context listens and auto-freezes the account.

Domain Events:

FraudDetected	Suspicious activity detected — account will be frozen

5 · Notification Context

Responsibility: Inform customers of important account actions. Pure event consumer — no commands are issued to this context.

Entity: Notification (stored in DB; simulates email/SMS in v1)

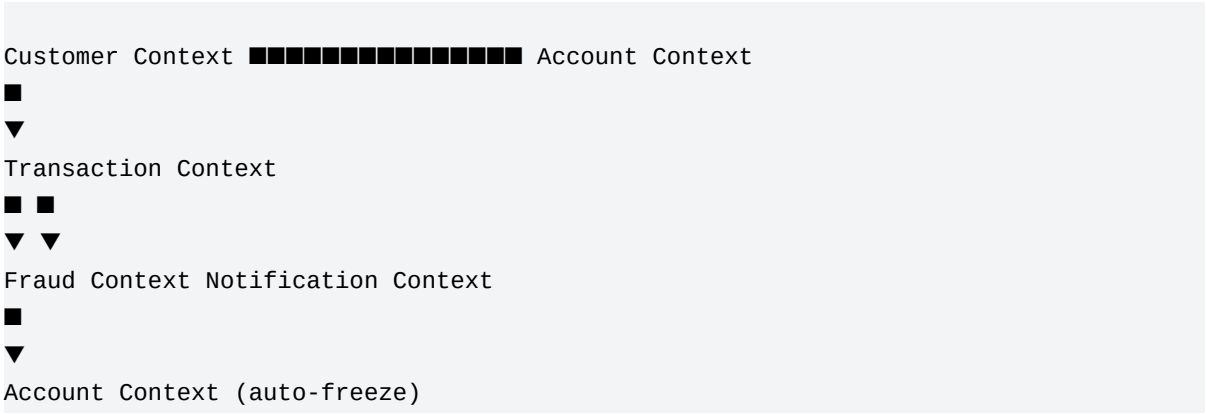
Event → Notification Mapping:

Event	Notification Message
AccountOpened	"Your account [IBAN] has been created"
MoneyDeposited	"You received $X into your account"
MoneyWithdrawn	"You withdrew $X from your account"
TransferCredited	"You received $X from [name]"
TransferDebited	"You sent $X to [name]"
TransferFailed	"Your transfer of $X has failed"
AccountFrozen	"Your account has been frozen"
FraudDetected	"Suspicious activity detected on your account"

Summary — All Contexts at a Glance

Context	Aggregate	Commands	Key Events
Identity	Customer	RegisterCustomer UpdateCustomer	CustomerRegistered CustomerUpdated
Account	BankAccount	OpenAccount FreezeAccount CloseAccount	AccountOpened AccountFrozen AccountClosed
Transaction	Transaction	Deposit Withdraw Transfer	MoneyDeposited MoneyWithdrawn TransferDebited TransferCredited TransferFailed
Fraud	FraudAlert	— (auto-triggered)	FraudDetected
Notification	Notification	— (event consumer)	—

Context Map (Event Flow)



Your Next Steps

1. Draw bounded context diagrams — show how events flow between contexts.

2. Design DB tables — one schema per context (they must NOT share tables).
3. Create aggregate diagrams — show entities, value objects, and relationships.
4. Share your diagrams and we'll review together before writing any code.
